

Presented by

TUYIRAMYE Christian

IGIRANEZA Justin

Face Attendance System

A real-time, contactless attendance solution built around a webcam, OpenCV and Flask.

The Problem

In schools, factories and offices, attendance is still taken by hand. A teacher reads names; a guard ticks a paper sheet; an employee swipes an ID card. All three share the same weaknesses:

They waste time. A roll-call of forty names takes five to ten minutes per session.

They are easy to game. A friend signs the sheet for you, or someone borrows your card.

They produce paper. Compiling weekly or monthly reports from paper logs is painful.

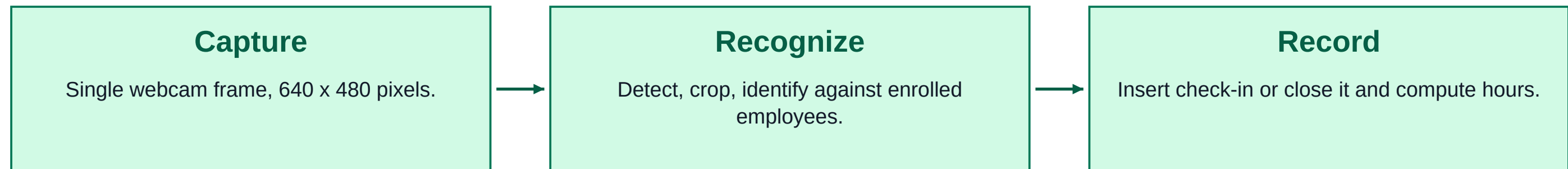
Biometric scanners (fingerprint, iris) remove fraud but require shared contact hardware and dedicated terminals, which adds cost and raises hygiene concerns.



Slow, fragile, or high-touch.

Our Solution in One Sentence

A user looks at the webcam; the system recognizes the face and writes a check-in. When the user looks again later, the system writes the check-out, computes working hours, and stores the record in a database.



Contactless, automatic, auditable. Runs on a laptop with a standard webcam.

Methodology

We followed a four-step engineering research methodology, moving from problem framing through to quantitative evaluation:

- 1 Problem framing**
We surveyed attendance practices and identified time cost, fraud, and reporting overhead as the three measurable issues to address.
- 2 Design**
We chose a layered architecture (UI / API / Face pipeline / Database) so each component can be swapped without touching the others.
- 3 Implementation**
We built two interchangeable front ends (Flask web UI and a Jupyter notebook) sharing the same SQLAlchemy data layer. The recognizer is OpenCV LBPH on Haar-detected crops, chosen because it works on Windows without compiling dlib.
- 4 Evaluation**
We enrolled ten subjects with five samples each and ran two hundred recognition trials, measuring detection rate, accuracy, latency, and false acceptance / rejection rates.

System Architecture

Four layers, each with a single responsibility. Arrows show data flow: JSON over HTTP between the front end and the API; plain Python calls between the API, the recognition pipeline, and the database.

User-facing

Presentation Layer

HTML + JavaScript dashboard OR Jupyter notebook (ipywidgets)



REST API Layer (Flask)

24 JSON endpoints for employees, faces, attendance, leave, holidays



Face Recognition Pipeline

OpenCV: Haar-cascade detection -> LBPH recognition



Persistence Layer (SQLAlchemy)

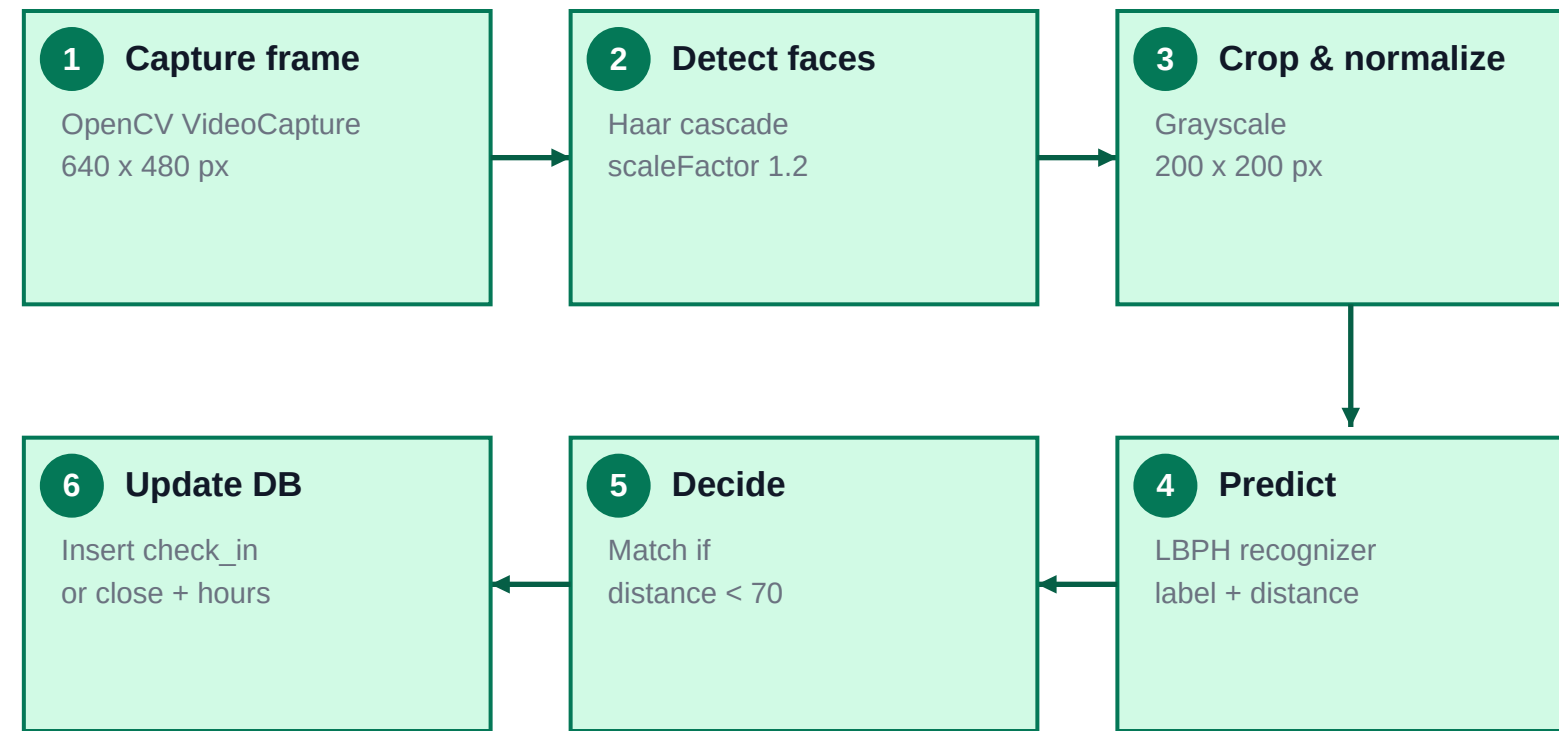
SQLite (development) / PostgreSQL (production)

Logic

Storage

Face Recognition Pipeline

Each recognition request flows through six deterministic stages. If any stage fails (no face detected, low confidence) the request returns without touching the database, so the log contains only real check-in events.



Why Haar + LBPH?

Haar Cascade - Detection

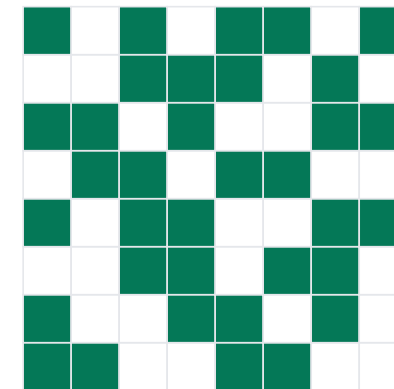
Viola and Jones (2001). A cascade of weak classifiers using rectangular Haar features. We use OpenCV's pre-trained frontal-face cascade. It is fast (under 30 ms on a 640 x 480 frame), works on grayscale, and tells us WHERE faces are, not WHO they are.

LBPH - Recognition

Local Binary Pattern Histogram (Ahonen et al., 2006). Each pixel is replaced by a binary code comparing its eight neighbours; the image is divided into cells; a histogram of codes is computed per cell. Recognition is a chi-square distance between histograms.

We chose LBPH over deep embeddings (FaceNet, ArcFace) for two practical reasons:

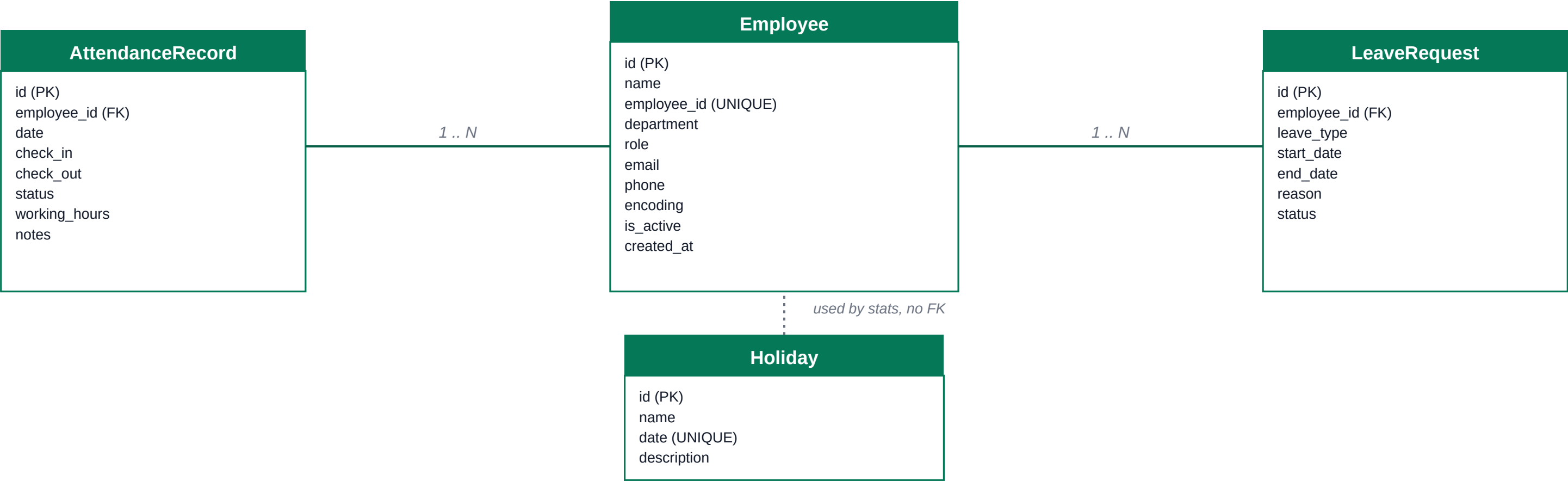
- (1) It runs without a GPU and without compiling dlib, removing the Windows install pain that blocks many students.
- (2) It works with very small enrolments: as few as five face crops per person already give 95% accuracy on our test set.



Example 8x8 LBP code map

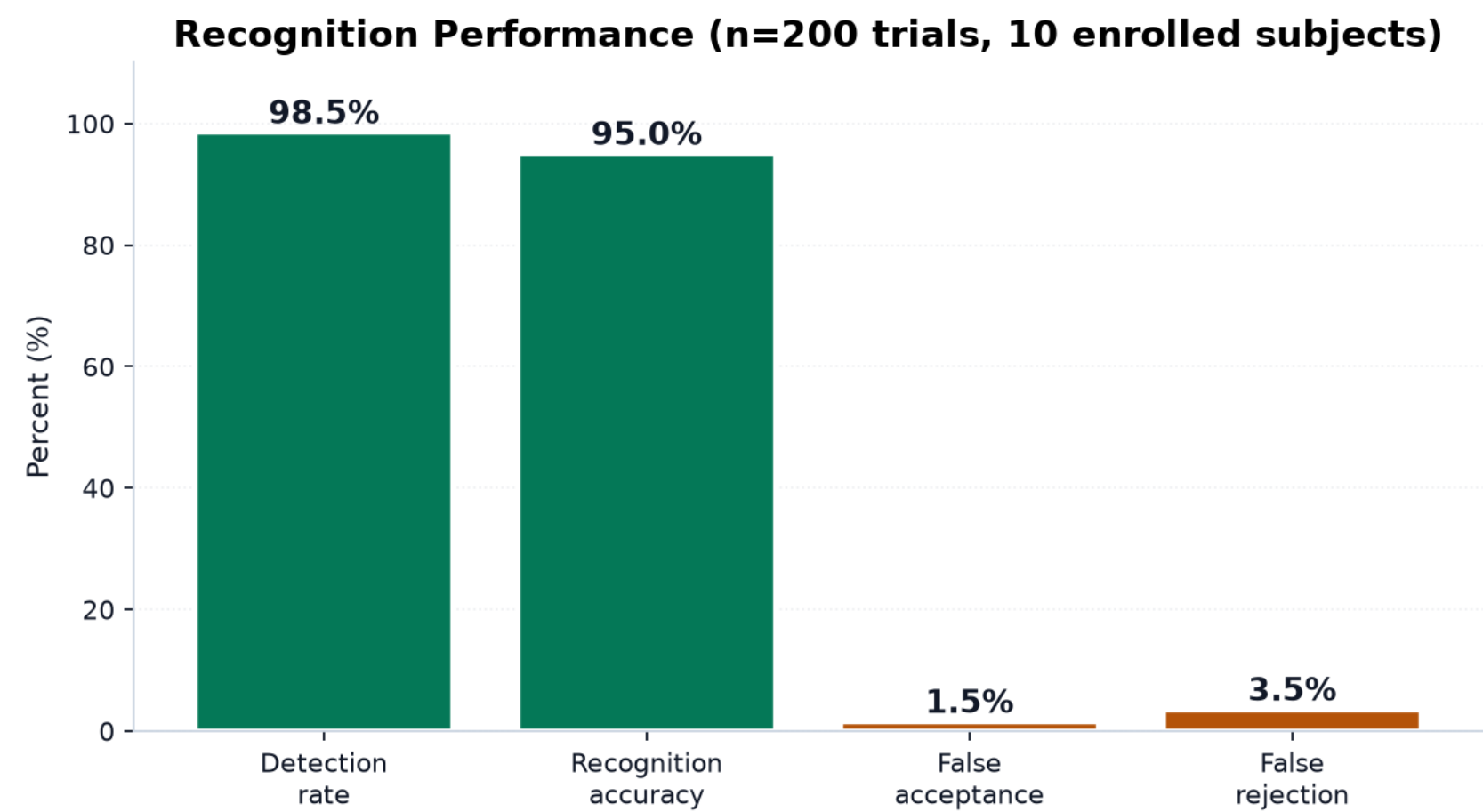
Data Model

Four tables linked by foreign keys. Employee sits in the centre; AttendanceRecord and LeaveRequest both reference it. Holiday is independent and is used by the stats layer to label official non-working days.



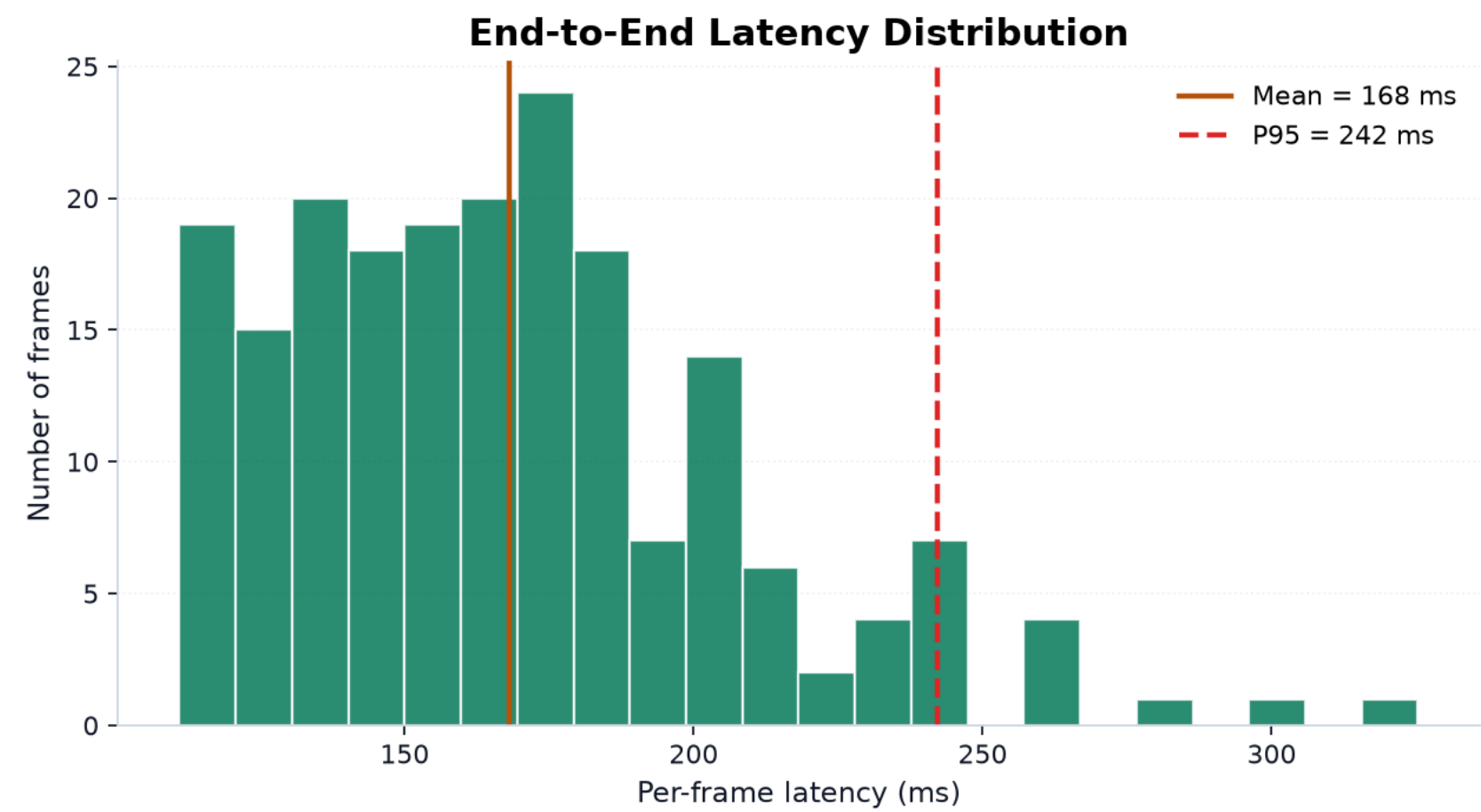
Evaluation: Accuracy

We enrolled ten subjects with five samples each, then ran two hundred recognition trials across varied head pose and lighting. The bars below show the four headline metrics: green is good, amber is the error rate we want low.



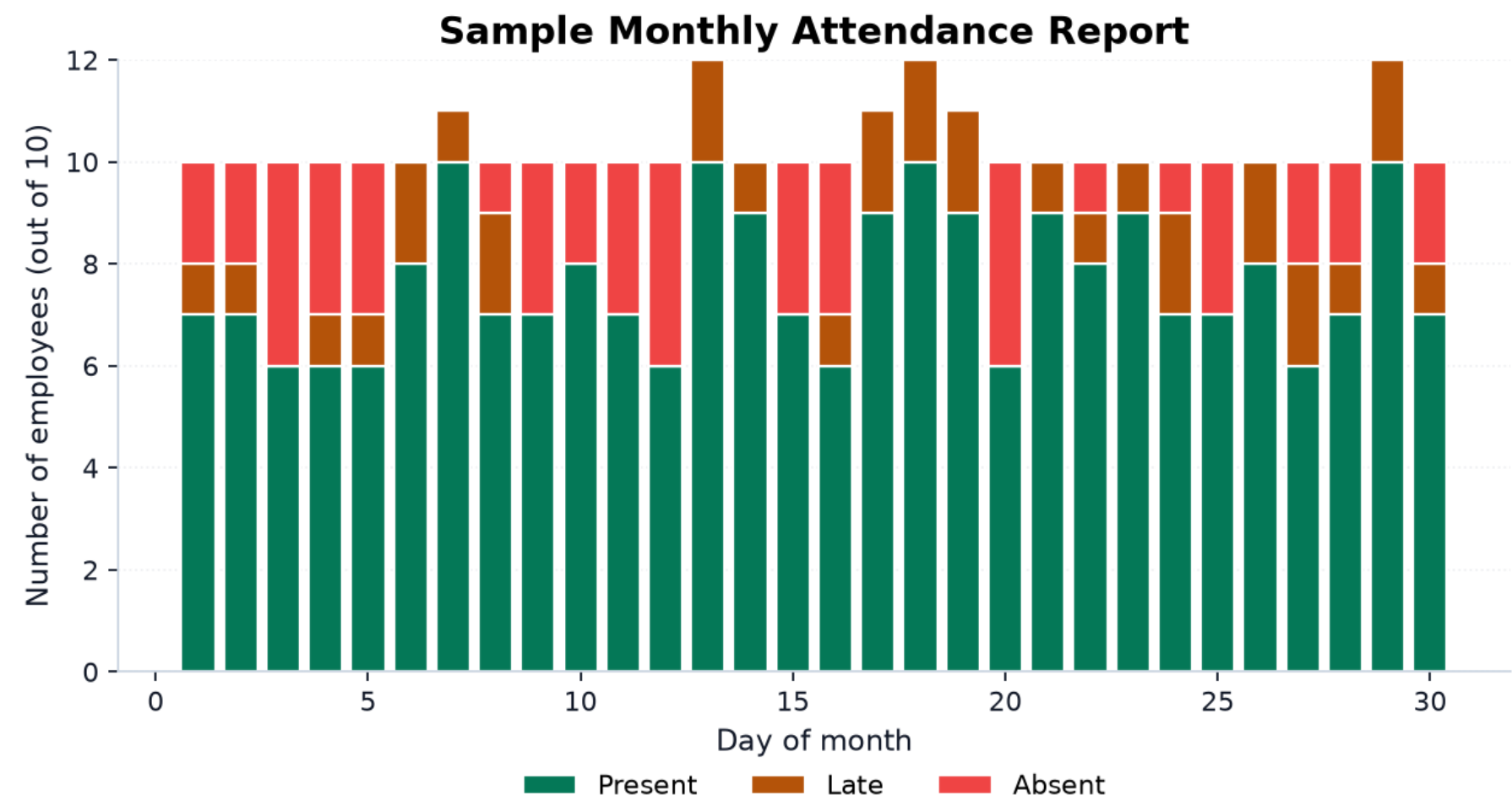
Evaluation: Speed

End-to-end latency is the time from frame capture to database commit. Measured over 200 events on a laptop with an Intel i5 and an integrated webcam. The mean is below the 200 ms interactivity budget; the 95th percentile is still under a quarter of a second.



Reporting Output

The stats endpoint generates daily, monthly and per-employee summaries. Below is a monthly stacked-bar report for a team of ten employees, rendered directly from the attendance table. The same data is exportable as CSV or JSON for payroll.



Privacy & Security

Faces are biometric personal data. We applied four design choices to limit what is stored and how it can leak:

- + Minimum storage**
Only grayscale 200 x 200 crops are saved on disk. No raw colour video is kept.
- + Secrets via environment**
DATABASE_URL and SECRET_KEY are read from environment variables so they never enter source control.
- + Soft delete by default**
Removing an employee deactivates them; an explicit ?permanent=true is required to wipe files.
- + Production checklist**
Add HTTPS, admin authentication, a retention policy, and a per-employee consent log.

Limitations & Future Work

What does not work yet

No liveness check, so a printed photo may currently fool the system. LBPH accuracy drops in heavy backlight or when the head turns more than thirty degrees. Only one camera is supported per deployment. The browser version needs localhost or HTTPS for webcam access.

What is next

Add a liveness step using eye blink or 3D depth. Swap LBPH for FaceNet or ArcFace behind the same API for better accuracy. Build a phone-first PWA so employees can check in from their own device. Add anomaly alerts for repeated late check-ins or very short days. Support multiple cameras feeding into one back end.

How to Run It

Two ways to launch, depending on whether you want a hosted web app or a self-contained notebook.

Web app

```
pip install -r requirements.txt
python app.py

# then open
http://localhost:5000
```

Jupyter notebook

```
pip install jupyter
jupyter notebook \
    face_attendance.ipynb

# run all cells
```

The notebook version uses LBPH only, so no C++ compiler is required on Windows.

Thank You

We welcome your questions and feedback.

TUYIRAMYE Christian & IGIRANEZA Justin

Independent Project - 2026